

Graph Game Theory: An Algebraic Approach for General Game Playing

Chaim Duchovny
Artificial Gladiator League

August 14, 2026

This paper is dedicated to my memory of my father and to three special children - Yuval, Shahar and Liri — three gold hearted children who were the sole inspiration for writing this work and are the best proof that there is still good in this world. When intelligent androids will walk upon the face of the earth someday in the future, they will know that they were created by good people.

Abstract

General Game Playing (GGP) requires AI agents to reason about arbitrary games described in the Game Description Language (GDL) under strict time constraints, with effectively no opportunity for offline training. Achieving competent play across arbitrary games is a key milestone toward Artificial General Intelligence. We propose Monte Carlo Vector Search (MCVS), an extension of Monte Carlo Tree Search that incorporates algebraic state analysis into a modified PUCT selection rule. Our central insight is that a broad class of finite positional games admits a computable algebraic structure, captured by the proposed *abc* model, which embeds game states as vectors in a Hilbert space. Within this space the strategic value of a position is measured by the geometric proximity of its weighted adjacency matrix to known winning configurations, using the Frobenius norm. The matrix representation is extracted in polynomial time and supplies a zone-guidance bias that steers search toward promising regions. Experiments on Breakthrough, English Draughts, Amazons, Reversi, Connect Four, Hex, Havannah, Y, Go and Los Alamos Chess show that the zone-guided variant of MCVS (without a neural network) achieves competitive or superior performance to strong UCT, RAVE and MAST baselines on the majority of these domains. The results indicate that a persistent database of algebraic state vectors provides a useful inductive bias that improves performance over strong stateless baselines under equal time constraints.

Contents

1	Introduction	4
2	Motivation	4
3	Related Work	5
4	The <i>abc</i> Model	5
4.1	Game Formalism	5
4.2	The <i>abc</i> Model: Formal Definition	6
4.2.1	Hilbert Vector Space	7
4.3	Connecting GDL-1 to <i>abc</i> model	8
4.4	State Embedding Construction	9
5	Algebraic Structure Analysis	11
5.1	The Weighted Adjacency Matrix	11
5.2	Group of Affine Translations	13
6	Zone Guidance	13
6.1	Mathematical Integration	14
7	Monte Carlo Vector Search (MCVS)	15
7.1	Stage 1: Board Embedding and State Representation	15
7.2	Database Architecture	15
7.3	Stage 2: Algorithm Flow	16
7.3.1	Main Loop	17
7.4	MCVS Move Selection	18
7.5	Update Statistics (Backpropagation)	18
7.6	Database Architecture and Management	19
7.6.1	Atomic Runners	19
7.6.2	Spider Agents	20
7.7	Implementation Notes	21
8	Experiment	22
8.1	Experimental Setup	22
9	Discussion	25
9.1	Where MCVS Excels	25
9.2	Where the Advantage Diminishes	25
9.3	Role of the Persistent Database and Background Agents	26
9.4	Limitations	26
9.5	Implications	27
10	Conclusion and Future Work	27

A	Algebraic Development of The abc Model	28
A.1	Static Placement	31
A.2	Case 2: Piece Movement and Multi Movement in a Turn	33

1 Introduction

Deep reinforcement learning systems such as AlphaGo and AlphaZero achieved super-human performance in Go, chess, and shogi through massive self-play on specialized hardware. These successes, however, rely on extensive game-specific training and computational resources. General Game Playing (GGP) [5, 6] poses a stricter challenge: an agent must play arbitrary games described at runtime in the Game Description Language (GDL), typically under a short startup period and a few seconds per move, with no prior training.

We address this setting with an algebraic approach. A broad class of finite, deterministic, perfect-information games can be embedded into a Hilbert space via the proposed *abc* model, in which states become vectors and legal moves become affine transformations. From this embedding we extract, in polynomial time, a weighted adjacency matrix (and a connectivity-matrix variant) that captures geometric and topological structure. Monte Carlo Vector Search (MCVS) then uses the matrix distance to known winning configurations as a zone-guidance bias inside a PUCT-style selection rule. The resulting agent maintains a persistent database of vector states across games and requires no neural network at test time.

The main contributions of this paper are:

1. The *abc* algebraic embedding of positional games into a Hilbert space.
2. A polynomial-time matrix representation (weighted and connected adjacency variants) together with a zone-guidance mechanism.
3. Monte Carlo Vector Search, an MCTS extension that exploits the algebraic embedding and succeeds in learning competent play without a neural network.
4. An empirical evaluation on Breakthrough, English Draughts, Hex5x5, Hex7x7, Havannah, Y, Connect Four, Reversi, Go9x9, G13x13 and Los Alamos Chess, showing that the zone-guided agent is competitive with or superior to strong UCT, MAST [7] and RAVE [8] baselines on the majority of these domains.

2 Motivation

Beyond its technical contributions, one of the aims of this work is to demonstrate that effective search algorithms for complex games can be designed in a transparent and accessible manner. By presenting an algorithm whose core mechanisms remain relatively simple and interpretable, we hope to encourage children, students and young researchers that developing original AI methods is within reach of everyone.

3 Related Work

Monte Carlo Tree Search (MCTS) [1, 2] still remains the dominant paradigm for sequential decision making in games. AlphaGo and AlphaZero [3, 4] demonstrated that pairing MCTS with deep neural networks trained by large-scale self-play can produce superhuman play, but these systems require extensive offline training and game-specific computation.

General Game Playing (GGP) [5, 6] imposes a stricter constraint: an agent must play arbitrary games described at runtime in GDL under short time limits and without prior training. Existing GGP agents typically rely either on hand-crafted heuristics or on learned neural features that demand substantial data. To our knowledge, no prior GGP work extracts polynomial-time algebraic structure from game states and uses the resulting geometric similarity as an explicit bias inside a PUCT-style selection rule. Monte Carlo Vector Search addresses this gap by embedding states into a Hilbert space via the *abc* model and guiding search with matrix distances to historically successful configurations.

4 The *abc* Model

4.1 Game Formalism

Our discussion begins by introducing a special class of games that are amenable to our algebraic analysis. These are finite, deterministic, perfect-information games whose states can be embedded as vectors in a Hilbert space, as we will shortly demonstrate. This representation induces a computable weighted adjacency matrix in $\mathbb{C}^{n \times n}$.

Definition 4.1. *Positional Game*

A finite deterministic perfect game Γ is called Positional Game if every state $s \in \mathcal{S}$ can be identified uniquely by the following finite sequence:

$$(p_i, x_1^i, x_2^i, \dots, x_n^i)_{i=1}^N \subseteq P \times \mathbb{C}^n \quad (1)$$

where P is a finite set of pieces such that $\forall p_i \in P, p_i \in \mathbb{C}$.

From now on, when we denote a game by Γ , we say that Γ is a finite deterministic perfect game.

Definition 4.2. *Set of Game States*

Let Γ be a game and P a set of pieces. We define:

$$\mathcal{S} = \left\{ (p_i, x_1^i, x_2^i, \dots, x_n^i)_{i=1}^N \mid p_i \in P, (x_1^i, x_2^i, \dots, x_n^i) \in \mathbb{C}^n, \right\} \quad (2)$$

The set $\mathcal{S}$ is the set of all possible game states. Each $s \in \mathcal{S}$ captures a complete description of the board, pieces, and turn information at a given moment.

Remark 4.1. From definition 4.2 it follows that $|\mathcal{S}| < \infty$.

We begin by introducing the fundamental relation underlying our model:

$$x(t) = \prod_{i=1}^t B_i \cdot [I, B_1, B_2, \dots, B_t] \cdot c(0) \quad (3)$$

where

$$a(t) := \prod_{i=1}^t B_i \quad \text{and} \quad b(t) := [I, B_1, B_2, \dots, B_t].$$

Here, $a(t)$ represents the affine component, while $b(t)$ represents the board component. The matrices B_i encode the successive board cells or transformations, and $c(0)$ denotes the center of the board at the initial stage, $t = 0$. This decomposition motivates the name of the model.

Assuming that the relevant operators commute, we may define the state recursively by

$$c(t) := B(t) \cdot c(t-1), \quad t \geq 1, \quad c(0) := c_{\text{initial}}.$$

Consequently, we obtain the fundamental *abc formulation*, which provides a compact description of both a game state and its evolution:

$$\boxed{\begin{aligned} x(t) &= b(t) \cdot c(t), \\ c(t) &= B(t) \cdot c(t-1), \\ B(0) &= I, \\ c(0) &= c_{\text{initial}}. \end{aligned}} \quad (4)$$

A detailed derivation of the final relation is provided in the appendix.

4.2 The *abc* Model: Formal Definition

We now provide a formal definition of the *abc* model. This definition integrates the dynamic, algebraic, and graph-theoretic components of our framework into a single unified structure.

4.2.1 Hilbert Vector Space

To support algebraic operations, distance metrics, and topological analysis (such as clustering of winning positions), we embed the game states into a Hilbert space $\mathcal{H}_G$.

The choice of this vector space is motivated by the possibility of future research into quantum-mechanical methods for developing improved strategies, especially in games with imperfect information.

Although each concrete game Γ has a finite state space $\mathcal{S}$, the embedding is defined over an infinite-dimensional space that naturally generalizes to games of arbitrary size. Let $\mathcal{C} = \mathbb{C}^n \times \{1\}$ denote the space of homogeneous coordinates for a single cell.

The Hilbert vector game space is the infinite product:

$$\mathcal{H}_G = \prod_{i=1}^{\infty} \mathcal{C}^{k_i} = \underbrace{(\mathcal{C} \times \cdots \times \mathcal{C})}_{k_1 \text{ times}} \times \underbrace{(\mathcal{C} \times \cdots \times \mathcal{C})}_{k_2 \text{ times}} \times \cdots, \quad (5)$$

where k_i is the number of cells at level i in the *abc* construction.

Vector addition and scalar multiplication are defined component-wise:

$$(z_1^1, \dots, z_n^1, 1) \oplus (z_1^2, \dots, z_n^2, 1) = (z_1^1 + z_1^2, \dots, z_n^1 + z_n^2, 1), \quad (6)$$

$$z \odot (z_1, \dots, z_n, 1) = (z \cdot z_1, \dots, z \cdot z_n, 1), \quad z \in \mathbb{C}. \quad (7)$$

The inner product is given by

$$\langle u, v \rangle_{\mathcal{H}_G} = \sum_{i=1}^{\infty} \sum_{j=1}^n u_j^i \overline{v_j^i}. \quad (8)$$

For any finite game Γ , only finitely many terms in the sum are nonzero, so the inner product is well-defined and finite. This construction turns the set of game states into vectors in a Hilbert space, enabling the use of linear algebra tools, norm-based distances on weighted matrices, continuity arguments, and clustering proofs for winning, losing, and drawing regions.

Definition 4.3. We define a tokenized vector $\kappa(t, B_i) = (\kappa_1(t, B_i), \kappa_2(t, B_i), \dots, \kappa_d(t, B_i)) \in \mathbb{C}^d$ to encode piece attributes (occupancy, color, strategic value etc.) at each cell B_i .

Definition 4.4. The abc Model

Let Γ be a game, and let $\mathcal{H}_G$ be the associated Hilbert vector game space over $\mathbb{C}$ (constructed explicitly via the embedding ϕ in Section 4.3).

We define the *abc* model as the tuple

$$(M(t), \kappa(t, B), c(0), (SB, MB)),$$

where:

1. $M(t) = (V, E)$ is the level- t directed meta-game graph, with
 - (a) $V = \{b(t) \cdot c(t) \mid t \geq 0\}$ the set of level- t vector game graphs (vectors in $\mathcal{H}_G$);
 - (b) E the directed edges representing legal transitions from $b(t-1) \cdot c(t-1)$ to $b(t) \cdot c(t)$.
2. $\kappa(t, B)$ is the tokenized vector at level t (piecewise constant in the continuous formulation).
3. SB is the static board with initial center $c(0)$.
4. MB is the mobile board that evolves with each state transition.
5. $Node(t) = \{(\kappa_1, \kappa_2, x(t))\}$ where:

$$\kappa_1(B_i) = \begin{cases} 1 & \text{if } B_i \text{ is occupied a piece} \\ 0 & \text{otherwise} \end{cases} \quad (9)$$

$$\kappa_2(B_i) = \begin{cases} t_1 & \text{token of piece 1} \\ t_2 & \text{token of piece 2} \\ \vdots & \\ t_n & \text{token of piece } n \end{cases} \quad (10)$$

The model rests on the distinction between the fixed static board SB (on which moves are executed) and the dynamic mobile board MB (whose reference frame shifts via affine transformations).

4.3 Connecting GDL-1 to abc model

Having established our *abc* model we now construct an explicit embedding of GDL-1 positional game descriptions into our vector game space $\mathcal{H}_G$. We will demonstrate that:

1. Every game state can be represented as a vector in $\mathcal{H}_G$.
2. The *abc* model provides an injective mapping from states to vectors.
3. Game-theoretic properties are preserved under this embedding.

This establishes the relevance of our framework to practical GDL-1 games used in General Game Playing competitions. To achieve this, we prove two key results. Lemma 4.1 shows that every reachable GDL-1 game state admits a representation within the *abc* model, while Lemma 4.2 guarantees that this representation is injective, ensuring uniqueness of the embedded states.

Lemma 4.1. *GDL-1 to abc model connection:* *Every GDL – 1 program of finite perfect positional game Γ generates only finitely many reachable game states.*

Proof. Every GDL-1 game description D is the following set: $\{C, F, P\}$ where C is the set of all constants, F is the set of all function and P is the set of predicates. This set defines the vocabulary of D .

A finite game description in GDL-1 uses a finite vocabulary. Moreover GDL-1 forbids unrestricted function symbol nesting. Consequently the Herbrand Universe that can be generated from the set of D vocabulary is finite or more formally $H_D < \infty$.

Consequently, the Herbrand Base, denoted by H_B , is finite, that is $|H_B| < \infty$. It consists of all potential ground atoms, obtained by applying every predicate symbol $P \in D$ to all possible combinations of ground terms from the Herbrand Universe H_D .

Each game state in a GDL-1 description is represented by a finite set of ground true facts derived from the game rules. Since a GDL-1 program contains a finite set of constants, predicates, and stratified rules without unbounded function symbol nesting, the set of all possible ground atoms that can appear in true facts is finite. Consequently, the set of all distinct game states that can be represented and reached from the initial state via the next relation is finite.

□

4.4 State Embedding Construction

We want to construct a function ϕ such that:

$$\phi : S \longrightarrow \mathcal{H}_{\mathcal{G}}$$

Is well defined and injective.

Recall from Definition 4.2 a state $s \in S$ is the following sequence:

$$s = ((p_i, x_1^i, x_2^i, \dots, x_n^i))_{i=0}^N \subseteq P \times \mathbb{C}^n,$$

Let us begin from the first term in s until we reach the final term in the sequence s . We will define f to be as follows:

$$\begin{aligned}
\phi(s) = & \left(B_0 \cdot [I] \cdot c(0), \right. \\
& B_1 \cdot [I, B_1] \cdot c(0), \\
& \dots, \\
& \left. B_1 \cdot B_2 \dots \cdot B_n \cdot [I, B_1, \dots, B_n] \cdot c(0) \right)
\end{aligned} \tag{11}$$

The last equation is a set of vectors in $\mathcal{H}_{\mathcal{G}}$ which can be written as follows:

$$\phi(s) = (b(0) \cdot c(0), b(1) \cdot c(1), \dots, b(t) \cdot c(t)) = (b(i) \cdot c(i))_{i=0}^t \tag{12}$$

Let us define the following function ϕ_i as follows:

$$\phi(s) = (b(i)c(i))_{i=0}^t \tag{13}$$

We now prove the following lemma:

Lemma 4.2. *Injective Embedding:* *Let $\phi : S \rightarrow \mathcal{H}_{\mathcal{G}}$ defined in equation 13.*

Let S be as in Definition 4.2 and Γ as in Definition 1. Then ϕ is injective: $s_1 \neq s_2$ implies $\phi(s_1) \neq \phi(s_2)$.

Proof. Let $s_1, s_2 \in S$ with $s_1 \neq s_2$. We will show that $\varphi(s_1) \neq \varphi(s_2)$.

Since $s_1 \neq s_2$, from Definition 1, either:

1. Different sequence lengths: $|s_1| \neq |s_2|$, or
2. Different elements at some position: $s_1[i] \neq s_2[i]$ for some i .

Case 1: If $|s_1| = N \neq K = |s_2|$, then $\varphi(s_1)$ has $N + 1$ *abc* components, while $\varphi(s_2)$ has $K + 1$ components. Hence $\varphi(s_1) \neq \varphi(s_2)$.

Case 2: If $|s_1| = |s_2|$ but $s_1[i] \neq s_2[i]$ for some i , then the i -th *abc* component differs due to different piece positions/types in the construction, so $\varphi(s_1) \neq \varphi(s_2)$ which leads that φ is injective. $\square$

5 Algebraic Structure Analysis

5.1 The Weighted Adjacency Matrix

We begin by formally constructing the weighted adjacency matrix, which serves as the foundational component of our proposed algorithm.

Recall that for any cell index i and feature index j , the tokenized vector component $\kappa_j(t, B_i)$ encodes specific state information. Without loss of generality, let us fix $j = 1$ to denote the occupancy indicator function.

We first define an isolated pieces set:

Definition 5.1. *Isolated Pieces Set*

Let Γ a game. Let π be a piece, $x(t)$ a vector game graph at level t and $k, K \in \mathbb{R}^+$. We define Isolated Pieces Set to be the following set:

$$\mathcal{I} = \{i \mid |\{j \neq i : k \leq \|B_i(t)(c(t-1)) - B_j(t)(c(t-1))\|_2 \leq K\}| = 0\} \quad (14)$$

$$\kappa_1(B_i) = \begin{cases} 1 & \text{if } B_i \text{ is occupied a piece} \\ 0 & \text{otherwise} \end{cases} \quad (15)$$

Definition 5.2. *Adjacency Matrix*:

Let Γ a game and a vector game graph $x(t)$ and k and K in $\mathbb{R}^+$ such that $0 < k < K$. Then the Adjacency Matrix of vector game graph $x(t)$ will be defined as follows:

$$A[i, j](t) = \begin{cases} 1 & \text{if two pieces are adjacent on the board} \\ 1 & \text{isolated piece} \\ 0 & \text{otherwise} \end{cases} \quad (16)$$

Definition 5.3. *Token Matrix* T Let Γ be a game and $x(t)$ a vector game graph. The Token Matrix is the diagonal matrix:

$$T(x(t)) = \text{diag}(T_1, T_2, \dots, T_n),$$

where:

$$T_i = \begin{cases} v(\kappa(t, B_i)) & \text{if } \kappa_1(B_i) = 1, \\ 0 & \text{otherwise.} \end{cases}$$

The function v maps the tokenized vector to a positive real number representing the strategic importance of the piece.

Algorithm 1 Construction of Token Matrix $T(x(t))$ Aligned with Adjacency

```

1: procedure CONSTRUCTTOKENMATRIX( $A(x(t))$ ,  $Node(t)$ ,  $\kappa$ )
2:    $n \leftarrow$  dimension of adjacency matrix  $A$  ▷ same size as T
3:   Initialize  $T \leftarrow \mathbf{0}_{n \times n}$  ▷ diagonal
4:   for each row index  $i = 1$  to  $n$  do
5:     Let  $B_i \leftarrow$  the cell corresponding to row  $i$  in  $A$  ▷ direct mapping from A's
       indexing
6:     Retrieve tokenized vector  $\kappa(t, B_i)$ 
7:     if  $\kappa_1(B_i) = 1$  then ▷ occupied
8:        $T_{i,i} \leftarrow v(\kappa(t, B_i))$  ▷ e.g. 2 for black, 3 for white, or node weight
9:     else
10:       $T_{i,i} \leftarrow 0$ 
11:    end if
12:  end for
13:  return  $T$ 
14: end procedure

```

Remark 5.1. The matrix T is constructed so that its row/column indices exactly match those of the adjacency matrix $A(x(t))$. That is, the token $T_{i,i}$ corresponds to the same cell B_i as row i (and column i) in A . This alignment guarantees that the product $W = A \times D \times T$ correctly scales the interactions by the token value of the target node.

Definition 5.4. Distance Matrix $D(x(t))$

Let Γ be a game and $x(t)$ a vector game graph. The Distance Matrix is defined as:

$$D[i, j](t) = \begin{cases} \|B_i \cdot c(t-1) - B_j \cdot c(t-1)\|_2 & \text{if } i \neq j \text{ and the cells are adjacent according to } A[i, j](t) = 1, \\ \|B_i \cdot c(t-1) - I \cdot c(t-1)\|_2 & \text{if cell } i \text{ or } j \text{ is isolated,} \\ 0 & \text{if } i = j \text{ (self-distance).} \end{cases}$$

where $d_{isolated}$ is a large positive constant (e.g., 10 or the diameter of the board) representing a weak or penalized connection for isolated pieces.

5.2 Group of Affine Translations

Consider the set of pure translation matrices in homogeneous coordinates:

$$B(\mathbf{v}) = \begin{bmatrix} I_n & \mathbf{v} \\ 0 & 1 \end{bmatrix}, \quad \mathbf{v} \in \mathbb{C}^n. \quad (17)$$

The ordinary matrix product of two such matrices is again a pure translation:

$$B(\mathbf{v}_1) B(\mathbf{v}_2) = B(\mathbf{v}_1 + \mathbf{v}_2). \quad (18)$$

Consequently the set

$$\mathcal{T} = \{B(\mathbf{v}) \mid \mathbf{v} \in \mathbb{C}^n\}$$

equipped with matrix multiplication forms an abelian group. The identity element is $B(\mathbf{0}) = I_{n+1}$ and the inverse of $B(\mathbf{v})$ is $B(-\mathbf{v})$. The group $(\mathcal{T}, \cdot)$ is isomorphic to the additive group $(\mathbb{C}^n, +)$.

Working with the translation vectors $\mathbf{v}$ instead of the full matrices reduces both memory consumption and arithmetic cost while preserving the group structure required by the *abc* model.

Definition 5.5. *Weighted Adjacency Matrix* *Following the last definitions we define the weighted adjacency matrix of a vector game graph $x(t)$ by:*

$$W(x(t)) = A(x(t)) \times D(x(t)) \times T(x(t)) \quad (19)$$

6 Zone Guidance

The motivation underlying the development of the *abc* model and MCVS is to design an AI algorithm whose learning process and decision-making mechanism can be analyzed and interpreted more transparently. In particular, the algorithm is intended to identify a path that maximizes its objective, which, in the context of the games considered here, is to achieve a win or, at minimum, to secure a draw.

From this perspective, our algorithm may be viewed as a decision-support mechanism, analogous to a GPS system, which provides the human operator with an interpretable sequence of actions leading toward a favorable position. More broadly, we aim to develop an AI system that captures certain aspects of human strategic reasoning

by identifying recurring patterns and using them to guide the search toward winning positions.

We measure the distance between two weighted adjacency matrices by the Frobenius norm

$$d_F(W, W_1) = \|W - W_1\|_F = \sqrt{\sum_{i,j} (W_{ij} - (W_1)_{ij})^2}.$$

Because all subsequent formulae treat a larger value as more similar, we convert the unbounded distance into a bounded similarity score:

$$\text{Sim}(W, W_1) = \frac{1}{1 + d_F(W, W_1)} \in (0, 1],$$

where $\text{Sim} = 1$ if and only if the two matrices are identical.

6.1 Mathematical Integration

The policy prior is obtained via a similarity-augmented softmax:

$$P(a) = \frac{\exp(\delta s_a)}{\sum_j \exp(\delta s_j)}, \quad (20)$$

where

$$s_a = \alpha \max_{W_t \in \mathcal{D}_{\text{win}}} \text{Sim}(W_a, W_t) - \beta \max_{W_t \in \mathcal{D}_{\text{loss}}} \text{Sim}(W_a, W_t) + \gamma \max_{W_t \in \mathcal{D}_{\text{draw}}} \text{Sim}(W_a, W_t). \quad (21)$$

The parameters $\alpha, \beta, \gamma, \delta > 0$ control the relative influence of winning, losing and drawing templates on the prior.

The final PUCT-style selection score remains:

$$\text{Score}(s, a) = Q(s, a) + c \cdot P(a) \frac{\sqrt{N(s)}}{1 + N(s, a)}. \quad (22)$$

This formulation ensures that the prior $P(a)$ is strongly biased toward positions similar to historically winning states, while heavily penalizing proximity to losing states.

The action value $Q(s, a)$ is the mean of the rewards R received from all simulations that passed through action a :

$$Q(s, a) = \frac{1}{N(s, a)} \sum_{i=1}^{N(s, a)} R_i$$

where the rollout reward R (evaluated at the end of each simulation at depth t) is the classical terminal outcome, normalised as follows:

$$R = \begin{cases} 1.0 & \text{if the agent wins,} \\ 0.5 & \text{if the game ends in a draw,} \\ 0.0 & \text{if the agent loses.} \end{cases}$$

7 Monte Carlo Vector Search (MCVS)

The Monte Carlo Vector Search (MCVS) algorithm combines the algebraic *abc* state representation with a persistent database of vector game graphs to guide Monte Carlo Tree Search.

7.1 Stage 1: Board Embedding and State Representation

Before the start of a game, the board is centered at the origin in homogeneous coordinates:

$$c(0) = (0, 0, 1)^T.$$

Each cell B_i corresponds to a fixed position vector at the center of the cell. Pieces are represented by tokenized vectors $\kappa(t, B_i) = (\kappa_1, \kappa_2, \dots)$, where $\kappa_1 \in \{0, 1\}$ denotes occupancy and $\kappa_2 \in \mathbb{R}$ represents the type of the piece.

7.2 Database Architecture

The system maintains a per-game database. For each game, the database stores a sequence of nodes:

$$\{\text{Node}_t\}_{t=0}^T,$$

where each Node_t contains:

- Vector positions of all pieces (persistent across moves).
- Tokenized vectors $\kappa(t, B_i)$. For simplicity we will use κ_1 and κ_2 .
- Action value $Q(s)$ and visit count $N(s)$.

Captured pieces retain their position vectors with $\kappa_1 = 0$. Newly placed pieces receive new position vectors according to the *abc* model.

7.3 Stage 2: Algorithm Flow

The core execution loop of Monte Carlo Vector Search (MCVS) is genuinely different from standard Monte Carlo Tree Search (MCTS).

Unlike classical MCTS, which builds a transient search tree using only statistics accumulated during the current game, MCVS maintains a persistent database of *vector game graphs* and their associated weighted adjacency matrices $W(t)$ across multiple games. This enables the algorithm to perform topological similarity retrieval: at each decision point, candidate moves are evaluated not only by their empirical value Q and visit count N , but by how closely the resulting configuration matches historically successful (or unsuccessful) states in the embedded algebraic space. In this way, MCVS combines on-line tree search with lifelong, geometry-aware case-based reasoning grounded in the *abc* model.

The algorithm proceeds as follows:

1. **Cold Start Phase.** At the beginning of a game the database is empty. MCVS falls back to standard UCT/MCTS. In parallel, n atomic runners populate the database with full game traces.
2. **Activation Threshold.** MCVS becomes active once the database contains at least one complete game.
3. **Exploitation of Winning Histories.** When winning records exist at depth t , MCVS prioritizes moves appearing in high-quality traces. If the opponent follows a known path, search continues along it. Otherwise, candidate moves are ranked by the similarity of their resulting weighted matrix W to stored winning states.
4. **PUCT-Guided Selection with Similarity Bias.** For each legal action a transitioning from level $t - 1$ to t , MCVS computes

$$\text{Score}(s, a) = Q(s, a) + c \cdot P(a) \frac{\sqrt{N(s)}}{1 + N(s, a)},$$

where $P(a)$ is derived from topological similarity to winning/losing/drawing templates (Zone Guidance). The *abc* embedding ensures spatial consistency via the distance matrix D .

5. **Value and Visit Integration.** Final selection balances the empirical action value $Q(s, a)$, visit count $N(s, a)$, and matrix similarity. Background atomic runners and spider agents continuously update the database.

This flow is summarized in Algorithm 2.

7.3.1 Main Loop

Algorithm 2 Monte Carlo Vector Search (MCVS) - Main Loop

```
1: procedure MCVS(Initial State  $s_0$ , game_id)
2:   INITIALIZEDATABASE(game_id)
3:   Launch Atomic Runners
4:   while Game not terminated do
5:     if Database has sufficient data then
6:        $a \leftarrow \text{MCVSSELECT}(s, \text{game\_id})$ 
7:     else
8:        $a \leftarrow \text{UCTSELECT}(s)$ 
9:     end if
10:    Apply action  $a \rightarrow$  successor state  $s'$ 
11:    SAVENODE(game_id,  $s'$ ,  $t$ )
12:     $R \leftarrow \text{SIMULATEFROM}(s')$  ▷ zone-guided reward
13:    UPDATESTATISTICS(path,  $R$ )
14:     $s \leftarrow s'$ 
15:     $t \leftarrow t + 1$ 
16:  end while
17: end procedure
```

7.4 MCVS Move Selection

Algorithm 3 MCVS Move Selection

```

1: procedure MCVSSELECT(Current State  $s_t$ , game_id)
2:   candidates  $\leftarrow$  LegalMoves( $s_t$ )
3:   bestScore  $\leftarrow -\infty$ , bestMove  $\leftarrow$  null
4:   for each action  $a_i$  do
5:      $s' \leftarrow$  ApplyMove( $s_t$ ,  $a_i$ )
6:      $\triangleright$  Compute weighted matrix using abc embedding
7:      $A' \leftarrow$  AdjacencyMatrix( $s'$ )
8:      $D' \leftarrow$  DistanceMatrix( $s'$ )
9:      $T' \leftarrow$  TokenMatrix( $s'$ )
10:     $W' \leftarrow A' \times D' \times T'$ 
11:     $sim_{win} \leftarrow \max \text{Sim}(W', W_t)$  for  $W_t \in \mathcal{D}_{win}$ 
12:     $sim_{loss} \leftarrow \max \text{Sim}(W', W_t)$  for  $W_t \in \mathcal{D}_{loss}$ 
13:     $P(a_i) \leftarrow \text{SimilarityAugmentedPrior}(sim_{win}, sim_{loss})$ 
14:     $score \leftarrow Q(s_t, a_i) + c \cdot P(a_i) \cdot \frac{\sqrt{N(s_t)}}{1+N(s_t, a_i)}$ 
15:    if score > bestScore then
16:      bestScore  $\leftarrow$  score
17:      bestMove  $\leftarrow a_i$ 
18:    end if
19:  end for
20:  spiderRecs  $\leftarrow$  SPIDERSEARCH(Database[game_id], current depth)
21:  return bestMove (boosted by spider high- $Q$ /high- $N$  recommendations)
22: end procedure

```

7.5 Update Statistics (Backpropagation)

Algorithm 4 Update Q and N (Backpropagation)

```

1: procedure UPDATESTATISTICS(Path, finalReward  $R$ )
2:   for each  $(s, a)$  in Path do
3:      $N(s, a) \leftarrow N(s, a) + 1$ 
4:      $Q(s, a) \leftarrow Q(s, a) + \frac{R - Q(s, a)}{N(s, a)}$ 
5:   end for
6: end procedure

```

7.6 Database Architecture and Management

The MCVS algorithm relies on a persistent per-game database that stores the complete history of vector game graphs, including positions, tokenized vectors, weighted adjacency matrices (or the data needed to compute them), action values $Q(s, a)$, and visit counts $N(s, a)$. The atomic runners are responsible for continuously inserting new game traces into winning, losing, and drawing partitions and the spider agents are responsible to retrieve the best action that maximizes the $Score(s, a)$.

Algorithm 5 Database Creation and Node Storage

```

1: procedure INITIALIZEDATABASE(game_id)
2:   Create new entry: games[game_id]  $\leftarrow$  empty list of Nodes
3:   Set board center  $c(0) \leftarrow (0, 0, 1)^T$ 
4: end procedure
5: procedure SAVENODE(game_id, current_state, t)
6:   Create new Nodet:
7:     positions  $\leftarrow$  homogeneous vectors for all pieces/cells
8:      $\kappa \leftarrow$  tokenized vectors for each cell
9:      $A, D, T \leftarrow$  Adjacency, Distance, and Token matrices (or raw data)
10:     $Q, N \leftarrow$  current action values and visit counts (if available)
11:    Append Nodet to games[game_id]
12: end procedure
13: procedure UPDATETERMINAL(game_id, final_outcome)
14:   Mark last Node as terminal
15:   Store outcome  $\in \{\text{win, loss, draw}\}$ 
16:   Trigger final backpropagation of  $Q$  values
17: end procedure
18: procedure QUERYBYLEVEL(game_id, level  $t$ )
19:   return all nodes at depth  $t$  (with high  $Q$  and  $N$  prioritized for spiders)
20: end procedure

```

7.6.1 Atomic Runners

Atomic Runners¹ operate in the background and are the primary source of database growth.

¹The name **Atomic Runners** draws inspiration from the video game *Atomic Runner* and from the two-stage structure of a conventional thermonuclear weapon. In such a weapon, a fission-based primary stage generates the extreme conditions required to initiate fusion in the secondary stage. The name therefore serves as a metaphor for agents that traverse the game space while accumulating database (DB) information for use by our AI system.

Algorithm 6 Atomic Runner (Background Agent)

```
1: procedure ATOMICRUNNER(Current State  $s$ , game_id)
2:   while Game ongoing do
3:     Run full MCTS simulation from  $s$  until terminal
4:     Compute final zone-guided reward  $R$ 
5:     Record full trace and outcome into Database
6:     UPDATESTATISTICS(trace path,  $R$ )
7:   end while
8: end procedure
```

7.6.2 Spider Agents

Spider agents² perform asynchronous exploration of the database to support real-time move selection.

Algorithm 7 Spider Search (Asynchronous Recommendation)

```
1: procedure SPIDERSEARCH(Database, current level  $t$ )
2:   In background thread:
3:     Query nodes at level  $t$  with high  $Q(s, a)$  and high  $N(s, a)$ 
4:     Rank them by vector / matrix similarity  $\text{Sim}(W, W_t)$  to current state
5:     Maintain ordered list of promising moves
6:     Return top recommendations to MCVSSELECT
7: end procedure
```

²The name **Spider** is inspired by the famous PageRank algorithm and by the image of a spider traversing a network. In our framework, the network consists of game states connected by state transitions. The *Spider* agent explores this state space to retrieve information for the AI system. To control the size of the state database, we prune nodes whose visit counts fall below a predetermined threshold.

7.7 Implementation Notes

The selection policy prioritizes states from historically winning vector game graphs, using $Q(s)$ as the primary criterion and $N(s)$ as tie-breaker. Background atomic runners and spider agents operate asynchronously, enabling continuous database population and real-time graph exploration during play.

This design maintains the algebraic integrity of the *abc* model while leveraging historical topological information through weighted adjacency matrices.

8 Experiment

8.1 Experimental Setup

All experiments were conducted under identical conditions to ensure a fair comparison. We evaluated Monte Carlo Vector Search (MCVS) against three strong baselines: plain UCT, MAST, and RAVE. Every agent was allocated a fixed thinking time of **1 second per move**. No opening books or endgame tables were used.

For each game variant we played **50 games** (25 games per colour) between MCVS and each baseline, for a total of 150 games per game variant. Colours were strictly alternated. The Elo rating was computed with a K -factor of 24, using a common pool that included all agents.

MCVS (We nickname our AI agent **ShiRi**)³ was run with two background *atomic runners* that continuously generated self-play traces from the live root and updated the persistent vector database. In addition, a single *Spider Agent* was invoked during move selection. The zone-guidance mechanism (matrix similarity to stored winning, losing, and drawing configurations) became active once the database contained at least one completed game. All other algorithmic parameters were held fixed across games; no game-specific tuning was performed.

The experiments were executed on an HP 290 G2 MT workstation equipped with an Intel Core i5-8500 processor (6 cores / 6 threads) and 16 GB of RAM, without GPU acceleration. The workstation was manufactured in the year of 2018. All agents were implemented in the same framework and compiled with identical optimisation flags, so that differences in performance can be attributed to the search algorithms rather than to implementation disparities.

We report both raw win rates (including the absolute number of wins out of 150 games) and Elo ratings. In addition, we present an ablation that compares the classic weighted adjacency matrix against the connectivity-matrix variant described in Definition 8.1.

A subset of the games in the batch was evaluated under two representations: first, using a weighted adjacency matrix, and second, using a connected weighted adjacency matrix. We hypothesize that the latter representation may yield superior performance for inherently connected games such as Y, Hex, and Havannah.

Definition 8.1 (Connected Weighted Adjacency Matrix). *Let $x(t)$ be a vector game graph under the abc embedding. The Connected Weighted Adjacency Matrix $C(x(t)) \in [0, 1]^{n \times n}$ is defined on pairs of cells occupied by the same player by the convex combination:*

³The name **ShiRi**, for our AI is dedicated to **Shiri** Bibas, whose fierce protection of her children against all odds—echoing the spirit of the character Ellen **Ripley**—served as the enduring inspiration for the name of our AI.

$$C_{ij} = w_\tau \tau_{ij} + w_\alpha \alpha_{ij} + w_\sigma \sigma_{ij} + w_\theta \theta_{ij} \quad (i \neq j), \quad (23)$$

where the positive weights sum to one and the four channels are:

- τ_{ij} (topological) equals 1 when i and j belong to the same connected component, otherwise a soft-influence kernel of the Euclidean distance d_{ij} that is raised to a constant bridge strength whenever i and j form a classic two-point bridge;
- α_{ij} (advancement) is a rank-dependent factor that is high when both pieces (or at least one) are advanced, multiplied by a spatial decay of d_{ij} ;
- σ_{ij} (support) equals 1 when i and j are adjacent and otherwise a soft spatial decay inside a fixed neighbourhood radius;
- θ_{ij} (threat) is high when at least one of the two pieces has a high progress rank and the cells are spatially close.

The diagonal carries an isolation penalty:

$$C_{ii} = \begin{cases} \lambda(r_i) & \text{if cell } i \text{ forms a singleton group,} \\ 1 & \text{otherwise,} \end{cases} \quad (24)$$

where $\lambda(r_i) \in (0, 1]$ is a mild, rank-aware penalty and r_i denotes the player-aware progress rank of cell i (higher = closer to the opponent's back rank).

All entries of C lie in $[0, 1]$. The matrix is obtained in time linear in the number of occupied cells after a standard flood-fill that identifies the connected components.

Table 1: Aggregate performance of MCVS against UCT, MAST and RAVE (1 s per move, 25 games per colour per pairing, Elo $K = 24$). Win rates are overall (150 games). Best baseline is shown for reference.

Game variant	MCVS win rate	MCVS Elo	Best baseline (agent)	Best baseline Elo	Notes
English Draughts 8×8	88.7% (133/150)	1515.5	RAVE (0.0%)	1165.2	Dominant
Amazons	78.0% (117/150)	1450.5	RAVE (2.0%)	1256.0	Dominant
Connect Four 7×8	74.7% (112/150)	1312.6	RAVE (60.0%)	1369.6	Strong
Hex 5×5 connected	76.0% (114/150)	1337.9	RAVE (34.0%)	1233.8	Strongest
Hex 5×5 classic	74.0% (111/150)	1326.4	RAVE (32.0%)	1198.8	Strong
Breakthrough 8×8 classic	67.3% (101/150)	1358.2	RAVE (48.0%)	1229.7	Clear win
Reversi 8×8	67.3% (101/150)	1289.1	MAST (52.0%)	1290.6	Competitive
Hex 7×7 connected	60.7% (91/150)	1213.2	RAVE (58.0%)	1311.5	Solid
Y connected	59.3% (89/150)	1219.4	RAVE (70.0%)	1346.6	Competitive
Havannah classic	51.3% (77/150)	1147.9	RAVE (76.0%)	1347.3	Mixed
Y classic	51.3% (77/150)	1108.7	RAVE (84.0%)	1386.9	Mixed
Havannah connected	50.7% (76/150)	1146.3	RAVE (74.0%)	1305.8	Mixed
Breakthrough 8×8 connected	50.0% (75/150)	1138.3	RAVE (80.0%)	1251.5	Classic better
Hex 7×7 classic	48.7% (73/150)	1162.5	RAVE (76.0%)	1283.9	Connected better
Go 9×9 classic	38.7% (58/150)	1271.7	UCT (78.0%)	1323.5	Weak
Go 9×9 connected	39.3% (59/150)	1274.0	UCT (74.0%)	1295.8	Weak
Los Alamos Chess	16.0% (24/150)	1048.7	MAST (82.0%)	1339.6	Failure mode

Table 2: Classic vs. connected matrix ablation (MCVS win rate / Elo).

Game	Classic	Connected
Breakthrough 8×8	67.3% / 1358.2	50.0% / 1138.3
Hex 5×5	74.0% / 1326.4	76.0% / 1337.9
Hex 7×7	48.7% / 1162.5	60.7% / 1213.2
Havannah	51.3% / 1147.9	50.7% / 1146.3
Y	51.3% / 1108.7	59.3% / 1219.4
Go 9×9	38.7% / 1271.7	39.3% / 1274.0

9 Discussion

The experimental results demonstrate that Monte Carlo Vector Search, guided by algebraic zone similarity, can provide a substantial advantage over strong statistical baselines under strict time constraints and without neural networks.

9.1 Where MCVS Excels

The strongest results appear in games whose strategic value is closely tied to geometric and topological structure. In English Draughts and Amazons, MCVS achieved dominant performance (88.7% and 78.0% win rates, respectively). Both games reward control of connectivity, mobility, isolation of pieces, and territory — precisely the features captured by the weighted and connectivity matrices derived from the *abc* embedding. Once the persistent vector database contains even a modest number of winning configurations, the zone-guidance term in the selection rule systematically steers search toward geometrically promising regions. The baselines (UCT, MAST, RAVE), which lack this cross-game geometric memory, are unable to match the resulting inductive bias under a 1-second-per-move limit.

Similar, though less extreme, advantages are observed in smaller Hex boards, Connect Four, and classic Breakthrough. In these domains the matrix representation continues to supply a useful prior that improves sample efficiency.

9.2 Where the Advantage Diminishes

On larger or more complex connection games (Hex 7×7 , Havannah, Y) the margin shrinks and results become mixed. Two factors appear responsible. First, the branching factor and state-space size grow rapidly, so that a fixed 1-second budget yields fewer informative simulations. Second, the current matrix features, while still informative, become coarser relative to the richness of the position. The classic-versus-connected ablation (Table 2) shows that the richer connectivity matrix helps on some of these

games (Hex 7×7 , Y) but not on others, indicating that feature design remains an important degree of freedom.

Los Alamos Chess constitutes a clear failure mode under the present experimental conditions. Two practical factors appear decisive. First, the agent struggled to accumulate a sufficiently rich and diverse vector database: the longer games and higher branching factor meant that background atomic runners produced fewer high-quality terminal traces within the available wall-clock time. Second, the 1-second-per-move budget yielded substantially fewer simulations than in the other domains, so the zone-guidance signal, even when present, had limited opportunity to influence the search tree.

These observations suggest a concrete research question: can the performance gap be closed by allocating a larger simulation budget and by improving the rate at which the persistent database grows (for example through more aggressive background runners, selective trace filtering, or hierarchical storage of matrices)? We hypothesise that the answer is affirmative — that the current algebraic features are not fundamentally inadequate for chess-like games, but simply require more data and more search effort to become effective. Testing this hypothesis is left for future work.

9.3 Role of the Persistent Database and Background Agents

A distinctive element of MCVS is the combination of two background atomic runners with a persistent vector database. The runners continuously generate self-play traces from the live root, allowing the database to accumulate winning, losing and drawing configurations across games. The Spider Agent provides an additional, lightweight similarity-based recommendation that can break near-ties. The rapid rise in performance after the database becomes active (as low as one completed game) underscores the value of explicit geometric memory in a general-game setting where offline training is forbidden.

9.4 Limitations

Several limitations should be acknowledged. The quality of zone guidance depends on the diversity and accuracy of the stored matrices; early games can therefore be noisy. The current matrix constructions, while polynomial-time, remain relatively simple and do not yet incorporate spectral or higher-order relational features. Finally, all experiments were conducted on fixed classic games rather than on arbitrary GDL descriptions under full tournament conditions; automatic extraction of the *abc* embedding from raw GDL remains future work.

Finally, all experiments were performed on modest hardware (Intel Core i5-8500, 6 cores / 6 threads, 16 GB RAM, no GPU acceleration). The combination of a short 1-second-per-move budget with concurrent background runners therefore constrained both the number of simulations available to the main search and the rate at which the vector

database could grow. Stronger hardware would likely improve absolute performance, particularly on longer games such as Los Alamos Chess.

9.5 Implications

Taken together, the results support the central thesis of this work: a broad class of finite positional games admits a computable algebraic structure that can be exploited as an inductive bias inside Monte Carlo search. When that structure aligns with the strategic demands of the game, MCVS achieves strong play with modest computational resources and no neural network. When the alignment is weak, the same prior can be neutral or harmful. This suggests a productive research direction in which richer algebraic features and selective hybridisation with lightweight learned components are explored, while preserving the interpretability and training-free character of the core approach.

10 Conclusion and Future Work

We presented an algebraic framework for general game playing that embeds game states into a Hilbert space and extracts geometric structure through weighted adjacency matrices in polynomial time. When this structure is integrated as zone guidance inside Monte Carlo Vector Search, the resulting agent achieves competitive or superior performance to strong UCT, MAST and RAVE baselines across multiple domains, often with substantially reduced search effort. These results demonstrate that explicit algebraic analysis can provide a resource-efficient and interpretable inductive bias for general game playing, and they establish a foundation for further research in algebraic approaches to game AI.

Several promising directions emerge from this study. First, incorporating spectral features of the weighted adjacency matrix (for example eigenvalues or leading eigenvectors) could capture deeper structural invariants; efficient approximations such as randomized SVD or power iteration would mitigate the cubic cost of exact eigendecomposition. Second, adaptive hybrid architectures that dynamically blend algebraic zone guidance with lightweight neural evaluations—via confidence-weighted switching or selective fallback—may improve robustness across a wider range of games. Third, extending the *abc* framework to imperfect-information settings, possibly through quantum-inspired formalisms for belief states, remains a significant open challenge. Finally, the geometric representations developed here may also prove useful beyond games, for instance as structured inductive biases in new classes of neural models for sequential or relational data.

References

- [1] R. Coulom. Efficient selectivity and backup operators in Monte-Carlo tree search. In *Computers and Games*, pp. 72–83. Springer, 2006.
- [2] L. Kocsis and C. Szepesvári. Bandit based Monte-Carlo tree search. In *International Conference on Machine Learning*, pp. 282–289, 2006.
- [3] D. Silver et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.
- [4] D. Silver et al. Mastering Chess and Shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01724*, 2017.
- [5] M. Genesereth, N. Love, and B. Pell. General Game Playing: Overview of the AAAI competition. *AI Magazine*, 26(2):62–72, 2005.
- [6] N. Love, T. Hinrichs, D. Haley, E. Schkufza, and M. Genesereth. General game playing: Game description language specification. Technical report, Stanford University, 2006.
- [7] Y. Björnsson and H. Finnsson. CADIAPLAYER: A simulation-based general game player. *IEEE Transactions on Computational Intelligence and AI in Games*, 1(1):4–15, 2009.
- [8] S. Gelly and D. Silver. Combining online and offline knowledge in UCT. In *Proceedings of the 24th International Conference on Machine Learning*, pp. 273–280, 2007.

A Algebraic Development of The abc Model

The discrete *abc* model describes game evolution through sequential transformations. We now derive the continuous formulation, bridging discrete game dynamics and differential equations.

Theorem A.1. *Discrete to Continuous Game Evolution*

Let Γ be a game with duration $s > 0$ and time partition $0 = t_0 < t_1 < \dots < t_n = s$ with uniform step size $\Delta t = t_{k+1} - t_k = s/n$.

The discrete game evolution is given by:

$$c_{k+1} = B_{k+1} \cdot c_k, \quad k = 0, 1, \dots, n-1 \quad (25)$$

where $c_k \in \mathbb{C}^{m+1}$ is the game state at time t_k , and $B_{k+1} = I + \Delta t \cdot A_{k+1}$ where A_{k+1} is a nilpotent matrix with $A_{k+1}^2 = 0$.

On each interval $[t_k, t_{k+1}]$, define the incremental transformation matrix:

$$B_{k+1}(\tau) = e^{(\tau-t_k)A_{k+1}} = I + (\tau - t_k)A_{k+1}, \quad \tau \in [t_k, t_{k+1}] \quad (26)$$

Then the continuous state evolution is given by:

$$\begin{cases} \frac{dc(\tau)}{d\tau} = A_{k+1} \cdot c(\tau), & \tau \in [t_k, t_{k+1}] \\ c(t_k) = c_k \\ c(\tau) = B_{k+1}(\tau) \cdot c_k = (I + (\tau - t_k)A_{k+1}) \cdot c_k \end{cases} \quad (27)$$

At discrete time points, continuous evolution agrees with the discrete equation:

$$c(t_{k+1}) = B_{k+1}(t_{k+1}) \cdot c(t_k) = e^{\Delta t \cdot A_{k+1}} \cdot c_k = B_{k+1} \cdot c_k = c_{k+1} \quad (28)$$

The incremental transformation matrix satisfies:

$$\frac{dB_{k+1}(\tau)}{d\tau} = A_{k+1} \cdot B_{k+1}(\tau), \quad \tau \in [t_k, t_{k+1}], \quad B_{k+1}(t_k) = I \quad (29)$$

Proof. Step 1: Structure of the transition matrix

By assumption, $B_{k+1} = I + \Delta t \cdot A_{k+1}$ where A_{k+1} is nilpotent with $A_{k+1}^2 = 0$.

Since $A_{k+1}^2 = 0$, the matrix exponential series terminates:

$$e^{\sigma A_{k+1}} = I + \sigma A_{k+1} + \frac{\sigma^2 A_{k+1}^2}{2!} + \dots = I + \sigma A_{k+1} \quad (30)$$

At the end of the interval ($\sigma = \Delta t$):

$$e^{\Delta t \cdot A_{k+1}} = I + \Delta t \cdot A_{k+1} = B_{k+1} \quad (31)$$

Step 2: Continuous evolution within $[t_k, t_{k+1}]$

Define the incremental transformation:

$$B_{k+1}(\tau) = e^{(\tau-t_k)A_{k+1}} = I + (\tau - t_k)A_{k+1}, \quad \tau \in [t_k, t_{k+1}] \quad (32)$$

The continuous state function is:

$$c(\tau) = B_{k+1}(\tau) \cdot c_k = (I + (\tau - t_k)A_{k+1}) \cdot c_k \quad (33)$$

Differentiate with respect to τ :

$$\frac{dc(\tau)}{d\tau} = A_{k+1} \cdot c_k \quad (34)$$

To express this in terms of $c(\tau)$, note that from $c(\tau) = (I + (\tau - t_k)A_{k+1}) \cdot c_k$, we have:

$$c_k = (I + (\tau - t_k)A_{k+1})^{-1} \cdot c(\tau) = (I - (\tau - t_k)A_{k+1}) \cdot c(\tau) \quad (35)$$

where we use the inverse formula for nilpotent matrices. Verification:

$$(I + \sigma A_{k+1})(I - \sigma A_{k+1}) = I - \sigma^2 A_{k+1}^2 = I \quad (36)$$

when $A_{k+1}^2 = 0$.

Substituting back:

$$\frac{dc(\tau)}{d\tau} = A_{k+1} \cdot (I - (\tau - t_k)A_{k+1}) \cdot c(\tau) = A_{k+1} \cdot c(\tau) - (\tau - t_k)A_{k+1}^2 \cdot c(\tau) \quad (37)$$

Using $A_{k+1}^2 = 0$:

$$\boxed{\frac{dc(\tau)}{d\tau} = A_{k+1} \cdot c(\tau)} \quad (38)$$

with initial condition $c(t_k) = c_k$.

Step 3: Differential equation for the transformation matrix

Differentiate $B_{k+1}(\tau) = I + (\tau - t_k)A_{k+1}$ with respect to τ :

$$\frac{dB_{k+1}(\tau)}{d\tau} = A_{k+1} \quad (39)$$

Using the matrix exponential property:

$$\frac{d}{d\tau} e^{(\tau - t_k)A_{k+1}} = A_{k+1} \cdot e^{(\tau - t_k)A_{k+1}} = A_{k+1} \cdot B_{k+1}(\tau) \quad (40)$$

Therefore:

$$\boxed{\frac{dB_{k+1}(\tau)}{d\tau} = A_{k+1} \cdot B_{k+1}(\tau)} \quad (41)$$

with boundary condition $B_{k+1}(t_k) = e^{0 \cdot A_{k+1}} = I$.

At the end of the interval: $B_{k+1}(t_{k+1}) = e^{\Delta t \cdot A_{k+1}} = B_{k+1}$ (the discrete transition matrix).

Step 4: Global cumulative transformation

The state at time $\tau \in [t_k, t_{k+1}]$ is:

$$c(\tau) = B_{k+1}(\tau) \cdot c(t_k) \quad (42)$$

The state at t_k is the result of k previous discrete transitions:

$$c(t_k) = B_k \cdot B_{k-1} \cdots B_1 \cdot c_0 = \prod_{j=1}^k B_j \cdot c_0 \quad (43)$$

Therefore, the global evolution is:

$$c(\tau) = B_{k+1}(\tau) \cdot \prod_{j=1}^k B_j \cdot c_0 = B(\tau) \cdot c_0 \quad (44)$$

where:

$$B(\tau) = B_{k+1}(\tau) \cdot \prod_{j=1}^k B_j, \quad \tau \in [t_k, t_{k+1}] \quad (45)$$

At $\tau = t_0$: $B(t_0) = B_1(t_0) = I$ since the product is empty when $k = 0$.

For $\tau \in (t_k, t_{k+1}]$ with $k \geq 1$: $B(\tau)$ is the product of all completed transitions up to time t_k , followed by partial evolution in the current interval.

Step 5: Verification of discrete agreement

At the discrete time t_{k+1} :

$$c(t_{k+1}) = B_{k+1}(t_{k+1}) \cdot c(t_k) = e^{\Delta t \cdot A_{k+1}} \cdot c_k = B_{k+1} \cdot c_k = c_{k+1} \quad (46)$$

This confirms that the continuous evolution exactly matches the discrete recurrence at all time points $\{t_k\}_{k=0}^n$.

Summary: Within each interval $[t_k, t_{k+1}]$, the game state evolves continuously according to $\frac{dc}{d\tau} = A_{k+1}c(\tau)$, with the nilpotent structure of A_{k+1} ensuring exact exponential solutions. The incremental transformation $B_{k+1}(\tau)$ satisfies $\frac{dB_{k+1}}{d\tau} = A_{k+1}B_{k+1}(\tau)$ with $B_{k+1}(t_k) = I$. The global transformation $B(\tau)$ accumulates all prior discrete transitions and matches the discrete equation at boundary points, with $B(t_0) = I$ as the unique baseline. $\square$

A.1 Static Placement

We derive the *abc* model for the static placement and piece movement variation. Let:

$$v_1 = (x_1, x_2, \dots, x_n, 1)^T, \quad v_2 = (y_1, y_2, \dots, y_n, 1)^T,$$

and let the initial center be

$$c(0) = (0, 0, \dots, 0, 1)^T.$$

Define the translation matrices

$$B_1 = \begin{bmatrix} I_n & \mathbf{v}_1 \\ 0 & 1 \end{bmatrix}, \quad B_2 = \begin{bmatrix} I_n & \mathbf{v}_2 \\ 0 & 1 \end{bmatrix},$$

where

$$\mathbf{v}_1 = (x_1, x_2, \dots, x_n)^T, \quad \mathbf{v}_2 = (y_1, y_2, \dots, y_n)^T.$$

Then:

$$v_1 = B_1 \cdot c(0), \quad v_2 = B_2 \cdot c(0).$$

Applying the same transformation again gives:

$$w_1 = B_1 \cdot v_1 = B_1^2 \cdot c(0), \quad w_2 = B_2 \cdot v_2 = B_2^2 \cdot c(0).$$

Cross-application yields:

$$u_1 = B_2 \cdot w_1 = B_2 \cdot B_1^2 \cdot c(0), \quad u_2 = B_1 \cdot w_2 = B_1 \cdot B_2^2 \cdot c(0).$$

Since translation matrices commute:

$$B_1 \cdot B_2 = \begin{bmatrix} I_n & \mathbf{v}_1 + \mathbf{v}_2 \\ 0 & 1 \end{bmatrix} = B_2 \cdot B_1.$$

Therefore:

$$u_1 = B_1^2 \cdot B_2 \cdot c(0), \quad u_2 = B_1 \cdot B_2^2 \cdot c(0).$$

Thus the recursive form becomes:

$$x(t) = \prod_{i=1}^t B_i \cdot [I, B_1, B_2, \dots, B_t] \cdot c(0)$$

A.2 Case 2: Piece Movement and Multi Movement in a Turn

Let a piece move from:

$$v_1 = (x_1, x_2, \dots, x_n, 1)^T$$

to

$$v_2 = (y_1, y_2, \dots, y_n, 1)^T.$$

Define the displacement vector

$$\mathbf{t} = (y_1 - x_1, y_2 - x_2, \dots, y_n - x_n)^T.$$

The translation matrix is:

$$T = \begin{bmatrix} I_n & \mathbf{t} \\ 0 & 1 \end{bmatrix}.$$

Then:

$$v_2 = T v_1.$$

For multiple transformations, write:

$$c_{k+1} = B_{k+1} \cdot c_k$$

with each movement represented by a matrix T_j or B_j depending on whether it is a move or placement.

For simultaneous movement of two pieces with translations T_a and T_b , the ordered update is:

$$b(2) \cdot c(2) = B_1 \cdot T_a \cdot T_b \cdot [I, B_1, (T_a, T_b)] \cdot c(0).$$

Using the lexicographic order rule, the transformations are applied in a fixed order so the model remains consistent.

Acknowledgments

This paper was refined with the assistance of artificial intelligence tools for grammar correction, English language polishing, and LaTeX formatting. All intellectual content,

theoretical development, mathematical derivations, experimental design, and final revisions were performed by the author. The author reviewed and takes full responsibility for all AI-assisted content.